

SISTEMA MULTIENTREPRISE DE GESTIÓN VETERINARIA

MANUAL TÉCNICO



LUIS ALMACHI, EMILIO LÓPEZ,
CRISTOPHER SANTANDER,
SEBASTIANGONZAGA, ANGEL GALARZA

TUTOR: M.SC ELVIS PACHACAMA
ITQ



ÍNDICE & TABLAS

MANUAL TÉCNICO DE ARQUITECTURA Y API REST	2
1. Información General del Proyecto	2
Tabla: Inf General	2
2. Arquitectura del Sistema y Directorios	2
2.1 Patrón Arquitectónico (MVC por Capas)	2
2.2. Estructura de Paquetes y Directorios	2
Tabla: Estructura	3
2.3. Stack Tecnológico y Componentes de Infraestructura	3
Tabla: Tecnologia	4
2.4. Flujo de Datos Transaccional	4
3. Seguridad y Aislamiento Multiempresa	5
Seguridad y Aislamiento Multiempresa (Tenants)	5
3.1. Mecanismo de Autenticación (JWT + BCrypt)	5
3.2. Aislamiento de Datos por Empresa (Multitenancy)	5
3.3. Control de Acceso Basado en Roles (RBAC)	6
4. Diseño de Base de Datos y Persistencia	7
Diseño de Base de Datos y Diccionario de Datos	7
4.1. Estrategia de Persistencia y Aislamiento	7
4.2. Diccionario de Datos (Entidades Clave)	7
Tabla: empresa	7
Tabla: usuario	8
Tabla: mascota	9
Tabla: producto	9
5. Especificación de Endpoints y Contrato de API	10
Control de Calidad y Reporte de Testing (QA)	10
5.1. Entorno de Ejecución y Herramientas	10
5.2. Resumen Metrológico de Pruebas	11
Tabla: Pruebas Realizadas	11
5.3. Matriz de Comportamientos Validados Correctamente	11
5.4. Registro y Matriz de Defectos (Bug Tracking)	12
Tabla: Bug Tracking:	12
6. Conclusiones	13
Conclusiones Técnicas	13
Recomendaciones para Despliegue en Producción (Trabajo Futuro)	13



MANUAL TECNICO (VETERINARIA) ARUITECTURA & API REST

MANUAL TÉCNICO DE ARQUITECTURA Y API REST Sistema de Gestión Veterinaria SaaS (Multiempresa)

1. Información General del Proyecto

Tabla: Inf General

Parámetro	Especificación Técnica
Backend	Java 17 LTS
Servidor de Aplicaciones	Apache Tomcat 10.1.55
Gestor de Construcción	Apache Maven
Base de Datos	PostgreSQL 18
Acceso a Datos	JDBC (Java Database Connectivity)
Manejo de JSON	Google Gson
Seguridad	JSON Web Tokens (JWT) + BCrypt Hashing

2. Arquitectura del Sistema y Directorios

2.1 Patrón Arquitectónico (MVC por Capas)

El backend utiliza una **Arquitectura en Capas orientada a Servicios RESTful**, adaptando el patrón clásico **Modelo-Vista-Controlador (MVC)**:

- **Modelo (Model & DAO):** Define la estructura de las entidades relacionales y gestiona la persistencia de datos.
- **Controlador (Servlet API):** Recibe las peticiones HTTP, gestiona los códigos de respuesta (200, 201, 400, 404, 409) y delega el procesamiento.
- **Vista (Frontend Desacoplado):** No forma parte del servidor Java; la interfaz corre en el cliente y consume la API enviando/recibiendo JSON.

2.2. Estructura de Paquetes y Directorios

El proyecto está empaquetado mediante Apache Maven bajo el nombre **VETERINARIAITQ**. La separación de clases en Java se organiza en los siguientes paquetes:





Tabla: Estructura

Paquete / Carpeta	Nombre de Capa / Componente	Descripción y Responsabilidad Principal
com.itq/config/	Configuración Base	ConexionBD.java: Manejo y apertura de conexiones JDBC con PostgreSQL.
com.itq/dao/	Capa de Acceso a Datos (DAO)	Consultas SQL puras (operaciones CRUD) ejecutadas mediante JDBC.
com.itq/dto/	Objetos de Transferencia (DTO)	ApiResponse.java: Manejo estandarizado de respuestas HTTP (exito, mensaje, datos).
com.itq/exception/	Gestión de Excepciones	Captura centralizada y estructurada de errores del servidor.
com.itq/filter/	Filtros y Seguridad HTTP	Filtros CORS e interceptores para autenticación de Tokens JWT.
com.itq/model/	Modelos / Entidades (POJOs)	Clases Java que representan exactamente las 22 tablas relacionales de la base de datos.
com.itq/service/	Lógica de Negocio	Servicios, validaciones del sistema y aislamiento de datos para Multiempresa.
com.itq/servlet/	Controladores REST	Servlets que exponen Endpoints HTTP para procesar peticiones GET, POST, PUT y DELETE.
com.itq/util/	Utilidades General	Funciones auxiliares: generación y validación de JWT, y hashing de contraseñas con BCrypt.

2.3. Stack Tecnológico y Componentes de Infraestructura

Tabla: Tecnologia

Capa	Tecnología	Justificación Técnica
------	------------	-----------------------



Lenguaje Base	Java 17 LTS	Versión de soporte extendido con alta estabilidad y compatibilidad.
Servidor de Aplicaciones	Apache Tomcat 10.1.55	Contenedor ligero compatible con las especificaciones de Jakarta Servlets.
Gestión de Proyecto	Apache Maven	Gestión de dependencias y compilación automatizada en archivo .war.
Base de Datos	PostgreSQL 18	Motor relacional robusto con soporte nativo para funciones JSON y UUIDs.
Acceso a Datos	JDBC Nativo	Conexión directa y ligera con la BD, evitando el overhead de ORMs complejos.
Mapeo de JSON	Google Gson	Serialización de objetos Java hacia formato JSON y viceversa.
Seguridad	JWT + BCrypt	Autenticación mediante tokens en cabeceras y encriptado seguro de contraseñas.

2.4. Flujo de Datos Transaccional

1. **Petición del Cliente:** El frontend o Postman envía una solicitud HTTP a un endpoint expuesto por un Servlet (ej. POST /api/empresas).
2. **Interceptación y Seguridad:** El paquete `filter` captura la petición, verifica la validez del token JWT en la cabecera `Authorization` y aplica las políticas de CORS.
3. **Lógica de Negocio:** El Servlet transmite los datos a la capa `service`, la cual extrae el `id_empresa` para aplicar el aislamiento multiempresa y ejecuta las reglas de negocio.
4. **Persistencia:** La capa `dao` recibe la entidad del paquete `model` y ejecuta la sentencia SQL sobre PostgreSQL mediante `ConexionBD` (JDBC).
5. **Respuesta Estandarizada:** El resultado es empaquetado en un DTO (`ApiResponse`), convertido a JSON con la librería `Gson` y retornado con el código de estado HTTP correspondiente.



3. Seguridad y Aislamiento Multiempresa

Seguridad y Aislamiento Multiempresa (Tenants)

Esta sección documenta la implementación de la capa de seguridad, autenticación *Stateless* y las reglas de segregación de datos entre clínicas veterinarias en el backend VETERINARIAITQ.

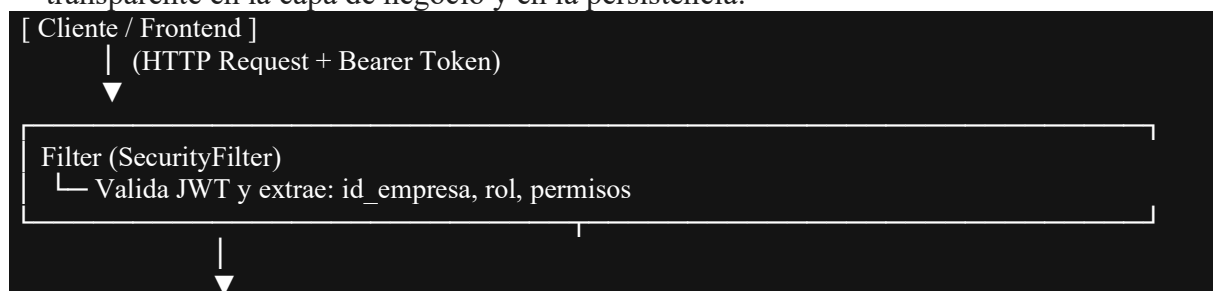
3.1. Mecanismo de Autenticación (JWT + BCrypt)

La aplicación elimina el manejo de sesiones en memoria en el servidor, adoptando un esquema de tokens **JSON Web Token (JWT)**.

- **Gestión de Contraseñas:** El sistema almacena las claves mediante algoritmos de hashing seguro con **BCrypt**. En la base de datos se registra únicamente el hash correspondiente.
- **Flujo de Autenticación:**
 - El usuario envía sus credenciales al endpoint de login.
 - La capa service consulta al dao correspondiente, valida la existencia del usuario y compara el hash BCrypt.
 - Si la verificación es exitosa, el backend genera un token JWT firmado que empaqueta las propiedades del usuario: `id_usuario`, `id_empresa`, `rol` y lista de permisos.
 - El frontend almacena dicho token y lo adjunta en cada petición HTTP mediante el encabezado:

3.2. Aislamiento de Datos por Empresa (Multitenancy)

El aislamiento de la información entre clínicas veterinarias se ejecuta de forma transparente en la capa de negocio y en la persistencia:





Service Layer

└─ Inyecta id_empresa automáticamente en la consulta



DAO Layer (JDBC)

└─ SELECT * FROM mascota WHERE id_empresa = ?

- **Extracción Automática:** El filtro de seguridad intercepta la petición, descodifica el token JWT y extrae el identificador de la clínica (id_empresa).
- **Inyección en Consultas SQL:** El módulo no requiere que el cliente seleccione o envíe manualmente la empresa en el cuerpo de las peticiones. Los servlets y DAOs filtran las consultas SQL (WHERE id_empresa = ?) utilizando la identidad extraída del token.

3.3. Control de Acceso Basado en Roles (RBAC)

La autorización del sistema se gestiona a través de una matriz de roles y permisos granulares:

- **SuperUsuario (SuperAdmin):**
 - Administra la totalidad de empresas/clínicas del sistema.
 - Gestiona el atributo activo de la tabla empresa.
 - Si una clínica es desactivada por el SuperUsuario, los interceptores del backend bloquean automáticamente el acceso a todos los usuarios vinculados a dicho id_empresa.
- **Farmacéutico:**
 - Rol con permisos acotados para operar sobre las entidades: *Categorías, Clientes, Facturación, Inventario y Productos*.
- **Usuarios de Clínica (Veterinario / Recepcionista):**
 - Su acceso está estrictamente delimitado a las entidades asociadas a su id_empresa.
- **Respuestas de Error de Seguridad:**
 - 401 Unauthorized: Devuelto cuando la petición carece del encabezado Authorization o el token JWT ha expirado/es inválido.



- 403 Forbidden: Devuelto cuando el usuario autenticado intenta acceder a un recurso sin los permisos requeridos para su rol.

4. Diseño de Base de Datos y Persistencia

Diseño de Base de Datos y Diccionario de Datos

Esta sección documenta la persistencia de datos implementada en **PostgreSQL 18** para la base de datos `aplicacion_veterinaria`, la cual consta de 22 tablas relacionales. El diseño garantizó la integridad referencial y el aislamiento multiempresa mediante claves foráneas y tipos de datos normalizados.

4.1. Estrategia de Persistencia y Aislamiento

- **Motor de BD:** PostgreSQL 18.
- **Identificadores Únicos:** Uso de UUID (`gen_random_uuid()`) como clave primaria (PK) en las tablas transaccionales principales para prevenir la predicción de IDs y facilitar la migración entre entornos.
- **Discriminador Multitenant:** Propagación de la clave foránea `id_empresa` (UUID) en todas las entidades transaccionales para habilitar el filtrado directo a nivel de consultas SQL.

4.2. Diccionario de Datos (Entidades Clave)

Tabla: empresa

Almacena la información de cada clínica veterinaria registrada en la plataforma.

Campo	Tipo de Dato	Restricciones	Descripción
<code>id_empresa</code>	UUID	PRIMARY KEY	Identificador único del tenant.
<code>ruc</code>	VARCHAR(13)	NOT NULL, UNIQUE	Registro Único de Contribuyentes.



razon_social	VARCHAR(150)	NOT NULL	Nombre legal o comercial de la clínica.
direccion	VARCHAR(255)	NULLABLE	Dirección física de la sucursal.
activo	BOOLEAN	DEFAULT TRUE	Estado operativo del tenant.

Tabla: usuario

Registra el personal que accede al sistema con credenciales.

Campo	Tipo de Dato	Restricciones	Descripción
id_usuario	UUID	PRIMARY KEY	Identificador del usuario.
id_empresa	UUID	FK -> empresa(id_empresa)	Vinculación directa con el tenant.
id_rol	INTEGER	FK -> rol(id_rol)	Rol asignado (RBAC).
usuario	VARCHAR(50)	NOT NULL, UNIQUE	Nombre de usuario para login.
clave_hash	VARCHAR(255)	NOT NULL	Contraseña encriptada con BCrypt.
nombres	VARCHAR(100)	NOT NULL	Nombres del usuario.
activo	BOOLEAN	DEFAULT TRUE	Estado del usuario.



Tabla: mascota

Almacena el registro de pacientes en el módulo clínico.

Campo	Tipo de Dato	Restricciones	Descripción
id_mascota	UUID	PRIMARY KEY	Identificador único del paciente.
id_empresa	UUID	FK -> empresa(id_empresa)	Discriminador multiempresa.
id_cliente	UUID	FK -> cliente(id_cliente)	Propietario de la mascota.
id_raza	INTEGER	FK -> raza(id_raza)	Clasificación por raza.
nombre	VARCHAR(100)	NOT NULL	Nombre de la mascota.
fecha_nacimiento	DATE	NOT NULL	Fecha de nacimiento.

Tabla: producto

Gestión de medicamentos, insumos e ítems comerciales de farmacia.

Campo	Tipo de Dato	Restricciones	Descripción
id_producto	UUID	PRIMARY KEY	Identificador del producto.
id_empresa	UUID	FK -> empresa(id_empresa)	Discriminador multiempresa.

id_categoria	INTEGER	FK -> categoria(id_categoria)	Categoría del producto.
id_iva	INTEGER	FK -> sri_iva(id_iva)	Porcentaje de IVA aplicable.
nombre	VARCHAR(150)	NOT NULL	Nombre comercial del ítem.
precio_unitario	DECIMAL(10,2)	NOT NULL	Precio de venta al público.
stock_actual	INTEGER	NOT NULL	Inventario físico existente.
stock_minimo	INTEGER	NOT NULL	Límite para disparo de alertas
fecha_caducidad	DATE	NULLABLE	Vencimiento del medicamento

5. Especificación de Endpoints y Contrato de API

Control de Calidad y Reporte de Testing (QA)

En esta sección se detallan las pruebas de verificación funcionales ejecutadas sobre el entorno de despliegue para validar la conectividad, la persistencia en la base de datos PostgreSQL, el empaquetamiento en Tomcat y las respuestas de la API REST.

5.1. Entorno de Ejecución y Herramientas

- **Servidor de Aplicaciones:** Apache Tomcat 10.1.55.
- **Base de Datos:** PostgreSQL 18 (Base: aplicacion_veterinaria, 22 tablas mapeadas).

- **Cliente de Pruebas REST:** Postman.
- **IDE y Entorno:** IntelliJ IDEA sobre Java 17 LTS.

5.2. Resumen Metrológico de Pruebas

Se ejecutó un plan de pruebas compuesto por **17 casos de prueba (CP)** sobre el módulo central de Mascotas y la conectividad base:

Tabla: Pruebas Realizadas

Métrica	Cantidad	Porcentaje
Casos de Prueba Ejecutados	17	100%
Pruebas Aprobadas (Exitosas)	13	76.5%
Defectos Detectados (Hallazgos QA)	4	23.5%

5.3. Matriz de Comportamientos Validados Correctamente

El sistema respondió de forma estable y alineada a los estándares REST en los siguientes aspectos:

- **Conexión a Base de Datos (CP-01):** Comunicación JDBC limpia contra PostgreSQL, identificando correctamente la existencia de las 22 tablas relacionales.
- **Despliegue del Servidor (CP-02):** Inicio correcto del contexto /VETERINARIAITQ en Apache Tomcat 10.1 sin excepciones de inicio.
- **Manejo de Respuestas HTTP Estandarizadas:**
 - 200 OK en consultas e impresiones de listas exitosas.
 - 201 Created con generación automática de identificadores UUID en altas.
 - 204 No Content en eliminaciones físicas ejecutadas en PostgreSQL.
 - 400 Bad Request ante UUIDs o identificadores con sintaxis inválida.

- 404 Not Found en intentos de consulta, actualización o eliminación de registros no existentes.
- 409 Conflict cuando se intenta registrar una mascota para un id_cliente que no existe en la base de datos (protección de integridad referencial).

5.4. Registro y Matriz de Defectos (Bug Tracking)

Se identificaron 4 hallazgos de severidad media orientados a la ausencia de validaciones de reglas de negocio en los endpoints de inserción y modificación.

Tabla: Bug Tracking:

ID	Módulo	Severidad	Comportamiento Obtenido (Fallo)	Comportamiento Esperado	Estado
DEF-01	Mascotas	Media	La API responde 201 Created al enviar un payload con nombre: "" (vacío).	400 Bad Request "El nombre es obligatorio".	Fixed
DEF-02	Mascotas	Media	La API permite guardar fechas de nacimiento futuras (ej. 2055-01-15) con 201 Created.	400 Bad Request "Fecha inválida".	Fixed
DEF-03	Mascotas	Media	El endpoint PUT permite actualizar registros dejando el nombre vacío ("") con 200 OK.	400 Bad Request Impedir actualización.	Fixed
DEF-04	Mascotas	Media	El endpoint PUT permite modificar la	400 Bad Request	Fixed



			fecha de nacimiento a un valor futuro con 200 OK.	Impedir actualización.	
--	--	--	---	------------------------	--

6. Conclusiones

Conclusiones Técnicas

1. **Solidez Arquitectónica:** El backend estructurado en Java 17 con servlets Jakarta, controladores DAO vía JDBC y arquitectura en capas garantiza una separación limpia entre la lógica de negocio, la persistencia relacional en PostgreSQL y la entrega del API.
2. **Seguridad y Multitenencia Efectiva:** La integración del modelo de autenticación mediante JSON Web Tokens (JWT) con hashing BCrypt permite un aislamiento *stateless* transparente entre clínicas veterinarias, asegurando que las consultas SQL filtren de manera automática los datos pertenecientes al `id_empresa` del usuario autenticado.
3. **Respuesta Transaccional:** El API demostró una integración adecuada de los códigos de estado HTTP ante errores de sintaxis y violaciones de clave foránea en la base de datos.

Recomendaciones para Despliegue en Producción (Trabajo Futuro)

1. **Implementación de Refresh Tokens:** Evitar el uso de tokens JWT con tiempos de expiración prolongados. Se sugiere manejar un *Access Token* de corta duración (ej. 15-30 minutos) y un *Refresh Token* almacenado de forma segura para renovar la sesión sin solicitar credenciales continuamente.
2. **Revocación de Sesiones:** Diseñar una lista negra (*blacklist*) en memoria (usando herramientas como Redis) para invalidar tokens JWT cuando un usuario cierre sesión o cuando un SuperUsuario desactive una veterinaria en tiempo real.



